Blocking plus MEC as One Procedure

Disjoint graph-based blocks, pooled unsupervised training, and blockwise greedy prediction

This report gives a single, fully specified method for combining the `blocking` package with maximum entropy classification (MEC). The method has four defining features:

- **graph-based disjoint blocks** are produced first by `blocking`;
- **training blocks** are then sampled *at random without replacement* until two thresholds are met: a minimum number of training pairs and a minimum lower bound on the number of nonmatches;
- a **single unsupervised MEC density-ratio model** is estimated on the union of the selected training blocks, using only within-block comparison pairs;
- the learned density-ratio model is **applied blockwise to all final blocks**, where the number of matches is estimated locally and the final classification set is obtained by the standard greedy MEC rule.

The internal mechanics of MEC itself are assumed known. The purpose of the document is to define precisely how blocking is admitted into MEC and how the resulting blocked pipeline can be implemented.

Date. April 14, 2026.
Main sources. Struzik and Beręsewicz (2026) for continuous MEC, the current `blocking` package manual and repository, and the current `automatedRecLin` documentation.

Contents

1 Scope and fixed design choices

We consider two files

$$A = \{a_1, \dots, a_{n_A}\}, \qquad B = \{b_1, \dots, b_{n_B}\},$$

and the unknown set of true one-to-one matches

$$M \subseteq A \times B.$$

The complete comparison space is

$$\Omega = A \times B.$$

The present note fixes one concrete design and does not discuss alternatives. The method is built under the following standing assumptions.

1. **The final blocks are disjoint.** After the blocking stage, the records are partitioned into disjoint bipartite blocks
$$(A_1, B_1), \dots, (A_L, B_L),$$
where the sets A_ℓ are pairwise disjoint and the sets B_ℓ are pairwise disjoint.
2. **MEC is trained once, then applied blockwise.** MEC is *not* re-estimated independently in every block. Instead, one common density-ratio model is fitted on a random subset of blocks and then reused in every block.
3. **Only within-block pairs are ever used.** Even during pooled training, no comparison pairs are formed across different original blocks. Pooling means a disjoint union of training blocks, not the creation of one cross-connected superblock.
4. **Prediction is local.** In every block, the number of matches is estimated locally and the final classification set is obtained by the usual greedy MEC heuristic.
5. **No household-network information is used.** The method relies only on MEC and blocking; no graph features, spectral embeddings, or household-level priors are added.

Under these assumptions the role of blocking is very clear. Blocking changes the comparison space from the full Cartesian product $A \times B$ to a union of disjoint local Cartesian products. The role of pooled training is equally clear: it stabilizes the estimation of the density ratio by using several blocks together, while preserving strictly blockwise prediction and final linkage decisions.

2 What `blocking` contributes to the pipeline

2.1 Package-level output

The current `blocking` package implements a graph-based blocking pipeline built from three stages: representation of records, approximate nearest-neighbor (ANN) search, and graph-based block formation. The package manual describes the main function `blocking()` as taking character strings or matrices, constructing representations such as shingles or vector-based features, applying one of several ANN backends, and creating blocks using graphs via `igraph` [2, 3]. The supported representation modes are `"shingles"`, `"custom_matrix"`, and `"vectors"`; supported ANN backends include `nnd`, `hnsw`, `annoy`, `lsh`, and `kd` [2]. The package also exposes a global ANN search-depth control `k_search` through `controls_ann()` [2].

The manual states that the return value of `blocking()` contains a table called `result` with columns `x`, `y`, `block`, and distance between points, as well as an optional `igraph` object [2]. In the present method, the only mandatory output from `blocking()` is the final block label. The returned ANN distance is not used in the current procedure.

2.2 Graph interpretation

For record linkage, the output of `blocking()` can be interpreted as an undirected bipartite graph

$$G = (V, E), \qquad V = A \cup B,$$

whose edges represent ANN candidate connections between records from the two files. The package uses graphs through `igraph` and assigns a block identifier to each returned candidate pair [2, 3]. Operationally, we interpret the block labels as connected-component labels of the ANN graph.

For each block label ℓ, define the record sets

$$A_\ell = \{a \in A : \exists b \in B \text{ such that } (a, b) \text{ is returned with block label } \ell\},$$

$$B_\ell = \{b \in B : \exists a \in A \text{ such that } (a, b) \text{ is returned with block label } \ell\}.$$

The operational MEC block is then

$$\Omega_\ell = A_\ell \times B_\ell.$$

Thus the ANN graph is used only to *define block membership*; once the blocks are fixed, MEC works on the full local Cartesian product inside each block.

2.3 Blocked comparison space

The final blocked comparison space is

$$\Omega^{\text{blk}} = \bigcup_{\ell=1}^{L} \Omega_\ell, \qquad \Omega_\ell = A_\ell \times B_\ell.$$

Because the blocks are disjoint on both files, this union is disjoint:

$$\Omega^{\text{blk}} = \bigsqcup_{\ell=1}^{L} \Omega_\ell.$$

If a true match falls across two different blocks, it is removed forever from the MEC stage. Denote the preserved within-block matches by

$$M_\ell = M \cap \Omega_\ell,$$

and the lost matches by

$$M_{\text{out}} = M \setminus \Omega^{\text{blk}}.$$

Then

$$M = \left(\bigsqcup_{\ell=1}^{L} M_\ell \right) \sqcup M_{\text{out}}.$$

If

$$R_{\text{blk}} = P\big((a, b) \in \Omega^{\text{blk}} \mid (a, b) \in M\big)$$

denotes block-level recall, then

$$\text{FNR}_{\text{blk}} = 1 - R_{\text{blk}}.$$

If $\text{MMR}_{\text{MEC|blk}}$ denotes the missing-match rate of MEC conditional on the match surviving blocking, then the overall missing-match rate satisfies

$$\text{MMR}_{\text{total}} = \text{FNR}_{\text{blk}} + (1 - \text{FNR}_{\text{blk}})\text{MMR}_{\text{MEC|blk}}. \tag{1}$$

This decomposition is part of the method specification: blocking is not a neutral preprocessing step but a stage that determines which true matches remain available to MEC.

3 Training blocks and the lower bound on nonmatches

The central practical problem is that `blocking` may produce many small disjoint blocks. Instead of enlarging or merging the final operational blocks, the proposed method resolves this issue by pooling a *randomly selected subset of blocks* for the *training phase only*. The final prediction phase still operates block by block.

3.1 Per-block counts

For each final block ℓ, write

$$a_\ell = |A_\ell|, \qquad b_\ell = |B_\ell|.$$

The total number of within-block pairs is

$$p_\ell = a_\ell b_\ell.$$

Under one-to-one linkage, the maximum possible number of true matches inside block ℓ is

$$m_\ell^{\max} = \min(a_\ell, b_\ell).$$

Therefore the number of nonmatches inside block ℓ is at least

$$u_\ell^{\min} = a_\ell b_\ell - \min(a_\ell, b_\ell). \tag{2}$$

Proposition 1 (Guaranteed lower bound on within-block nonmatches)**.** *For every block ℓ, the number of true nonmatches in $\Omega_\ell = A_\ell \times B_\ell$ is at least $u_\ell^{\min}$ given in (2).*

Proof. Because linkage is one-to-one, block ℓ can contain at most $\min(a_\ell, b_\ell)$ true matches. Since the total number of pairs in the block is $a_\ell b_\ell$, at least $a_\ell b_\ell - \min(a_\ell, b_\ell)$ pairs must be nonmatches. □

The importance of (2) is purely operational: it gives a computable lower bound on the number of nonmatches available for pooled unsupervised training, even though the true labels are unknown.

3.2 Training thresholds

Choose two user-specified thresholds:

$$P_{\min} > 0, \qquad U_{\min} > 0,$$

where:

- $P_{\min}$ is the minimum required number of within-block training pairs, and
- $U_{\min}$ is the minimum required lower bound on the number of nonmatches.

The training stage will be built from a random subset of blocks whose union satisfies both thresholds simultaneously.

3.3 Random block sampling rule

Let $\{1, \ldots, L\}$ be the set of all final blocks. Generate a random permutation

$$\pi = (\pi(1), \ldots, \pi(L))$$

of the block indices. For each $t = 1, \ldots, L$, define the cumulative selected set

$$\mathcal{T}_t = \{\pi(1), \ldots, \pi(t)\}.$$

Its cumulative pair count is

$$P(\mathcal{T}_t) = \sum_{s=1}^{t} p_{\pi(s)},$$

and its cumulative lower bound on nonmatches is

$$U^{\min}(\mathcal{T}_t) = \sum_{s=1}^{t} u^{\min}_{\pi(s)}.$$

The training-block set is defined by the first-hitting rule

$$t^* = \min\Big\{t \in \{1, \ldots, L\} : P(\mathcal{T}_t) \geq P_{\min} \text{ and } U^{\min}(\mathcal{T}_t) \geq U_{\min}\Big\}. \tag{3}$$

If the set on the right-hand side is nonempty, the final training-block set is

$$\mathcal{T} = \mathcal{T}_{t^*}.$$

If the set is empty, the procedure stops and reports that the training sample is infeasible under the current blocking and the current thresholds.

This is the entire block-selection rule. It is intentionally simple: blocks are drawn at random without replacement until both thresholds are met.

3.4 Training space

Once $\mathcal{T}$ has been selected, the pooled training space is the disjoint union

$$\Omega_{\mathcal{T}} = \bigsqcup_{\ell \in \mathcal{T}} \Omega_\ell.$$

No pair from different original blocks is ever added. Pooling means that all within-block pairs from the selected blocks are placed into one joint MEC training sample, nothing more.

4 Pooled unsupervised training of the common density-ratio model

4.1 Training objects

For each pair $(a, b) \in \Omega_{\mathcal{T}}$, construct the comparison vector

$$\gamma_{ab} = \left(\gamma^1_{ab}, \ldots, \gamma^K_{ab}\right)^\top$$

using the fixed set of comparison variables and comparison functions chosen for MEC. The current method does not constrain the internal MEC variant: the common density-ratio model may be binary, continuous-parametric, continuous-nonparametric, or another MEC-compatible construction, provided that it returns an estimated density ratio

$$\widehat{r}(\gamma) = \frac{\widehat{m}(\gamma)}{\widehat{u}(\gamma)}.$$

The details of that estimation are assumed known and are not repeated here.

4.2 What is learned in the training phase

The training phase produces one common object:

$$\widehat{r}(\gamma),$$

that is, one common density-ratio model for the entire blocked problem. The model is estimated on $\Omega_{\mathcal{T}}$ and then frozen. No new density-ratio model is re-estimated in the prediction phase.

If the training MEC fit returns an estimated number of matches on the training space,

$$\widehat{n}_{M,\mathcal{T}},$$

we additionally define the training-space candidate-match rate

$$\widehat{\pi}_{\mathcal{T}} = \frac{\widehat{n}_{M,\mathcal{T}}}{|\Omega_{\mathcal{T}}|}. \tag{4}$$

This quantity is used only to initialize the local fixed-point iteration in every block.

5 Blockwise prediction on all final blocks

After the pooled density-ratio model has been trained, prediction is performed *in every final block*, including the blocks that were used during training. The rule is uniform across all blocks.

5.1 Local scoring

Fix one block $\ell \in \{1, \dots, L\}$. Its local comparison space is

$$\Omega_\ell = A_\ell \times B_\ell, \qquad n_\ell = |\Omega_\ell| = a_\ell b_\ell.$$

For every pair $(a, b) \in \Omega_\ell$, compute the comparison vector γ_{ab} and evaluate the common fitted ratio

$$\widehat{r}_{\ell,ab} = \widehat{r}(\gamma_{ab}).$$

These are the local MEC scores used in the block.

5.2 Local estimation of the number of matches

The number of matches is estimated *locally* in each block through a block-specific fixed-point equation. Since the common density-ratio model $\widehat{r}(\gamma)$ has already been learned in the pooled training stage, only the local cardinality parameter remains to be determined at prediction time.

For block ℓ, define the local mapping

$$F_\ell(m) = \sum_{(a,b)\in\Omega_\ell} \min\left\{ \frac{m\widehat{r}_{\ell,ab}}{m(\widehat{r}_{\ell,ab} - 1) + n_\ell}, 1 \right\}, \qquad m \in [0, \min(a_\ell, b_\ell)]. \tag{5}$$

The local match count is then defined by the fixed-point equation

$$m = F_\ell(m). \tag{6}$$

Let $m_\ell^\star$ denote a solution of (6) in the admissible interval $[0, \min(a_\ell, b_\ell)]$. The local number of matches is

$$\widehat{n}_{M,\ell} = \min\{\lfloor m_\ell^\star \rceil, \min(a_\ell, b_\ell)\}, \tag{7}$$

where $\lfloor \cdot \rceil$ denotes rounding to the nearest integer.

Thus the blockwise cardinality-estimation stage is expressed directly through a local fixed-point equation. The essential point is that the score function $\widehat{r}(\gamma)$ is global, whereas the cardinality $\widehat{n}_{M,\ell}$ remains block-specific.

5.3 Greedy blockwise classification set

Once $\widehat{n}_{M,\ell}$ has been obtained, sort the pairs in Ω_ℓ in decreasing order of $\widehat{r}_{\ell,ab}$. Traverse the ordered list greedily. A pair (a, b) is accepted if and only if record a has not yet been used in the current block and record b has not yet been used in the current block. Continue until exactly $\widehat{n}_{M,\ell}$ pairs have been accepted. The resulting blockwise classification set is denoted by

$$\widehat{M}_\ell \subseteq \Omega_\ell.$$

The blockwise entropy-maximizing interpretation of this step is inherited from MEC and is assumed known. The present document fixes the greedy rule as the decision mechanism.

5.4 Global output

Because the blocks are disjoint on both files, the final global linkage output is simply

$$\widehat{M} = \bigsqcup_{\ell=1}^{L} \widehat{M}_\ell,$$

and the global estimated number of matches is

$$\widehat{N}_M = \sum_{\ell=1}^{L} \widehat{n}_{M,\ell}.$$

Proposition 2 (Blockwise separability of the final linkage)**.** *Under disjoint final blocks, the global greedy MEC output equals the disjoint union of the blockwise greedy outputs.*

Proof. No record from A_ℓ appears in any block $m \neq \ell$, and no record from B_ℓ appears in any block $m \neq \ell$. Therefore the one-to-one constraints are internal to each block and the greedy decision inside one block cannot affect feasibility in another block. The global linkage is thus the disjoint union of the blockwise linkages. □

6 The full method as a single algorithm

The whole proposed method can now be stated in a compact form.

Input

Two files A and B; a chosen `blocking()` configuration; a fixed MEC configuration (variables, comparison functions, and one chosen MEC variant); thresholds $P_{\min}$ and $U_{\min}$; and a random seed for block sampling.

Output

A final linkage set $\widehat{M} \subseteq A \times B$ and a final estimated number of matches $\widehat{N}_M$.

Algorithm

1. Run `blocking()` on A and B.

2. From the returned block labels, reconstruct the disjoint record blocks $(A_1, B_1), \ldots, (A_L, B_L)$.
3. For each block ℓ, define the local Cartesian product $\Omega_\ell = A_\ell \times B_\ell$ and compute
$$p_\ell = a_\ell b_\ell, \qquad u_\ell^{\min} = a_\ell b_\ell - \min(a_\ell, b_\ell).$$
4. Generate a random permutation of the block indices and select blocks without replacement until both cumulative thresholds are reached:
$$\sum_{\ell \in \mathcal{T}} p_\ell \geq P_{\min}, \qquad \sum_{\ell \in \mathcal{T}} u_\ell^{\min} \geq U_{\min}.$$
If no such set is reached before all blocks are exhausted, stop and declare the current training problem infeasible under the current blocking configuration.
5. Form the pooled training space
$$\Omega_{\mathcal{T}} = \bigsqcup_{\ell \in \mathcal{T}} \Omega_\ell.$$
Construct comparison vectors only for pairs in $\Omega_{\mathcal{T}}$.
6. Fit one unsupervised MEC density-ratio model on $\Omega_{\mathcal{T}}$ and obtain $\widehat{r}(\gamma)$ and $\widehat{\pi}_{\mathcal{T}}$.
7. For each block $\ell = 1, \ldots, L$:
   (a) construct comparison vectors for all pairs in Ω_ℓ;
   (b) evaluate the common score $\widehat{r}_{\ell,ab} = \widehat{r}(\gamma_{ab})$;
   (c) estimate the local number of matches $\widehat{n}_{M,\ell}$ by the local fixed-point iteration (6)–(7);
   (d) sort pairs in decreasing score order and build the greedy blockwise MEC set $\widehat{M}_\ell$ of size $\widehat{n}_{M,\ell}$.
8. Return
$$\widehat{M} = \bigsqcup_{\ell=1}^{L} \widehat{M}_\ell, \qquad \widehat{N}_M = \sum_{\ell=1}^{L} \widehat{n}_{M,\ell}.$$

This is the complete method.

7 Implementation notes

7.1 From package output to operational blocks

The package output is an ANN-derived candidate structure with block labels. The present method converts those block labels into record sets A_ℓ and B_ℓ, and then replaces the sparse ANN edge list by the full local Cartesian product $A_\ell \times B_\ell$. In other words, the ANN graph determines *where MEC may operate*, but MEC itself operates on all local pairs inside each final block.

7.2 Records with no candidate membership

A record that never appears in any returned candidate pair is not part of the blocked comparison space and is therefore excluded from MEC. Such records are treated as lost by the blocking stage, exactly as encoded by M_{out} and (1).

7.3 The role of ANN distance

The current method uses the `block` labels returned by `blocking()` but does not use the returned ANN distance `dist`. The distance is therefore ignored in both training and prediction.

7.4 External pre-stratification

The current `blocking` manual documents the arguments `on` and `on_blocking` as currently unsupported [2]. Therefore, if the user wishes to impose any coarse exact pre-stratification before graph-based blocking, that stratification must be created outside `blocking()`, and the entire pipeline described in this report must then be run separately inside each external stratum.

7.5 Reproducibility

Because the training-block set is sampled at random, the random seed is a formal part of the method definition. A run of the method is not fully specified unless the seed is fixed and recorded.

8 What the method achieves mathematically

The proposed method solves a concrete statistical and computational problem.

1. Blocking changes the comparison space from the infeasible full product $A \times B$ to the disjoint blocked space
$$\Omega^{\text{blk}} = \bigsqcup_{\ell=1}^{L} \Omega_\ell.$$
2. Random block sampling creates a training space that is large enough, by construction, in terms of both total pairs and guaranteed nonmatches.
3. Pooled unsupervised MEC training delivers one common density-ratio model
$$\widehat{r}(\gamma),$$
estimated only from within-block pairs drawn from the selected blocks.
4. Blockwise prediction preserves the local nature of linkage decisions. Each block receives its own estimated number of matches and its own greedy classification set.
5. Disjointness ensures that the final global linkage is simply the union of the local linkages.

In this sense, the method is a complete integration of graph-based blocking and MEC: blocking determines the admissible local comparison spaces, random pooled training determines one common ratio model, and blockwise MEC turns that common score into final one-to-one link decisions.

9 Conclusion

This report has specified one single blocked MEC method. The method uses `blocking()` to create graph-based disjoint blocks, samples training blocks at random until a minimum number of pairs and a minimum lower bound on nonmatches are both reached, estimates one common unsupervised MEC density-ratio model on the pooled union of those training blocks, and then

applies that model blockwise to every final block. In each block, the number of matches is estimated locally and the final classification set is produced by the usual greedy MEC rule.

The method is therefore fully defined by six objects:

(i) the blocking configuration,

(ii) the final disjoint blocks,

(iii) the threshold $P_{\min}$,

(iv) the threshold $U_{\min}$,

(v) the MEC configuration, and

(vi) the random seed.

Given these ingredients, the blocked MEC pipeline becomes a single reproducible procedure.

References

[1] Struzik, A. and Beręsewicz, M. (2026). Capturing Small Discrepancies in Record Linkage: A Maximum Entropy Framework with Continuous Similarity Measures. User-provided manuscript.

[2] Beręsewicz, M. and Struzik, A. (2026). `blocking`: Various Blocking Methods for Entity Resolution. CRAN manual, version 1.0.2, published 2026-03-11. https://cran.r-project.org/web/packages/blocking/blocking.pdf.

[3] Beręsewicz, M. and Struzik, A. (2026). `blocking` GitHub repository. https://github.com/ncn-foreigners/blocking.

[4] Struzik, A. and Beręsewicz, M. (2026). `automatedRecLin`: Record Linkage Based on an Entropy-Maximizing Classifier. Package documentation. https://ncn-foreigners.ue.poznan.pl/automatedRecLin/.

[5] Lee, D., Zhang, L.-C., and Kim, J. K. (2022). Maximum entropy classification for record linkage. *Survey Methodology*, 48(1).